

Smart Resume Analyzer with AI-Based Feedback

AI Capstone Project
Prepared by Mohammed Abdul Basith
SkillOrbit

Abstract

The Smart Resume Analyzer is an educational AI/NLP web application designed to help students evaluate and improve resumes. It accepts PDF or DOCX resumes, extracts their text, checks important resume sections, compares role-specific keywords, identifies missing skills and generates improvement suggestions. Results are presented through an interactive dashboard.

Introduction

Applicant Tracking Systems are commonly used to organize and screen applications. Students may not know which resume sections, technical skills and keywords are important for a target role. This project demonstrates a small, explainable prototype for resume analysis.

Problem Statement

Many students create resumes without understanding ATS requirements, industry-relevant technical skills, formatting standards, keyword optimization and role-specific expectations.

Objectives

Develop a web application that uploads resumes, extracts content, calculates a resume score, checks ATS compatibility, analyzes role-based skills, generates improvement suggestions and displays the results through a dashboard.

System Architecture

The user uploads a PDF/DOCX resume through the browser. The Flask backend receives the file, extracts text using PyMuPDF or python-docx, performs rule-based scoring and keyword analysis, stores the analysis summary in SQLite and returns structured results to the dashboard.

Module 1 — Resume Upload and Parsing

The application validates PDF/DOCX input and extracts textual content. This demonstrates file handling, document parsing and backend integration.

Module 2 — Resume Score Analyzer

The score uses structural checks for contact information, education, skills, projects and experience, together with text completeness and matched role skills. The approach is deliberately rule-based and explainable.

Module 3 — ATS Keyword Checker

The user selects Data Analyst, Web Developer, AI Engineer or Cloud Engineer. The system compares resume text with a predefined skill set and reports matched and missing skills.

Module 4 — Smart Feedback System

Suggestions are generated from missing sections, missing role skills and limited resume content. Examples include adding a technical skills section, including projects, improving contact details and adding relevant keywords.

Module 5 — Dashboard

The dashboard presents the resume score, ATS compatibility score, matched skills, missing skills, section checks and improvement suggestions.

Technology Stack

Frontend: HTML, CSS, JavaScript. Backend: Python Flask. AI/NLP: spaCy and rule-based keyword analysis. Document parsing: PyMuPDF and python-docx. Database: SQLite. The project can be deployed using platforms such as Render.

Testing

Testing covers valid PDF/DOCX uploads, unsupported file types, missing files, role selection, keyword matching and display of results. The included sample resume is designed to exercise the main analysis paths.

Results

The prototype provides a repeatable workflow from resume upload to scoring, keyword matching and feedback. Because the scoring is heuristic, the output is intended as educational guidance rather than a real hiring decision.

Limitations

The prototype does not train a deep-learning model, does not use an enterprise authentication system and does not perform semantic understanding of every resume phrase. Exact or phrase-based keyword matching can miss equivalent terminology.

Future Scope

Future versions could add semantic embeddings, better skill/entity extraction, job-description upload, resume version comparison, explainable score breakdowns and secure authenticated storage.

Conclusion

The project demonstrates how document processing, basic NLP, rule-based AI logic, web development and dashboard visualization can be combined into an industry-oriented prototype for resume analysis.